RAG: Chat With Your Documents

A step-by-step guide for your local Qwen + Python project

Beginner-friendly study guide • Concepts, examples, and practical exercises

1. What RAG means

RAG stands for Retrieval-Augmented Generation. It combines a search step with a language model: first retrieve relevant pieces of information, then ask the model to answer using those pieces.

Without RAG, a model may not know your private or recently updated documents. RAG does not retrain the model; it supplies relevant context at question time.

2. RAG pipeline overview

```
PDF / TXT / DOCX files\n        ↓\nExtract text and metadata\n        ↓\nClean and split into chunks\n        ↓
```

Ingestion is the preparation pipeline for files. Query time is the process that runs each time the user asks a question.

3. Document extraction

Extraction converts a file into text the program can process. PDF text extraction works best for digitally generated PDFs. Scanned PDFs may need OCR (optical character recognition). DOCX files contain paragraphs and other structured content; TXT files are already plain text.

Preserve metadata such as filename, page number, section heading and document ID. Metadata makes it possible to cite where an answer came from.

4. Cleaning and chunking

Cleaning removes irrelevant artifacts such as repeated whitespace or broken headers, while avoiding accidental deletion of meaningful content. Chunking divides long text into smaller passages.

Chunk size: amount of text in a chunk. Overlap: repeated text shared by neighboring chunks to preserve context at boundaries. Small chunks can lose context; large chunks can retrieve too much unrelated text. Test different settings on your own documents.

5. Embeddings and vector stores

An embedding model maps each chunk into a vector. A vector store or index keeps those vectors alongside text and metadata. At query time, the question is embedded using the compatible embedding model and similar chunks are retrieved.

Examples of vector search tools include FAISS for a local index and Chroma for a local or hosted database. The best choice depends on scale, filtering, persistence, operations and deployment needs.

6. Retrieval and ranking

The retriever selects passages likely to answer the question. Similarity search is useful, but can return false positives. A reranker can evaluate a smaller set of candidates more carefully. Hybrid search combines semantic retrieval with keyword matching.

Do not automatically assume the top result is correct. Tune the number of retrieved chunks, similarity thresholds, metadata filters and ranking strategy using test questions.

7. Grounded generation and citations

Build a prompt that clearly separates instructions, retrieved context and the user question. Tell the model to admit when the context does not contain the answer. Keep source metadata so the UI can display document names and page or section numbers.

Source citations should point to the actual retrieved passage. A citation is not proof of correctness unless the cited text genuinely supports the claim.

8. Build it incrementally in your existing project

• Keep your working app.py unchanged at first.

• Create a separate RAG module or a small test script.

• Start with one simple TXT file before adding PDF and DOCX extraction.

• Print extracted text and verify it before chunking.

• Inspect several chunks and their metadata.

• Test retrieval by printing the top matching chunks before calling Qwen.

• Only after retrieval works, connect retrieved context to the model prompt.

• Finally, add source citations to the Gradio UI and test empty or irrelevant questions.

9. Example chunking function

```
def split_text(text, chunk_size=500, overlap=80):\n    if chunk_size <= 0:\n        raise ValueError('chunk_siz
```

This simple example splits by characters, not tokens or semantic boundaries. It is intended to teach the concept; production systems often split by tokens, paragraphs, headings or document structure.

10. Quality checklist

• Can the extractor read the files correctly, including tables or scanned pages where needed?

• Are chunks neither too small nor too large for the content?

• Does retrieval return the correct passage for known questions?

• Does the answer avoid unsupported claims when evidence is missing?

• Are source names and page references correct?

• Are secrets excluded from Git and are private files protected?

• Are latency, memory use and failure cases tested on your Windows CPU setup?

11. Troubleshooting

No text extracted: the PDF may be scanned or image-based; inspect it and consider OCR.

Wrong chunks retrieved: inspect chunk boundaries, embedding model, top-k and metadata.

Correct context, wrong answer: improve the prompt, simplify context, compare model capability and add tests.

Slow on CPU: start with a small document collection, batch ingestion, and avoid re-embedding unchanged files.

Repeated sources or duplicates: track document IDs and content hashes; remove or update old indexed versions.

12. Next improvements

After a basic local RAG system works, consider persistent indexes, document update detection, page-aware citations, hybrid search, reranking, evaluation datasets, access controls, and then a FastAPI endpoint. Add one capability at a time and preserve a working baseline.

Official documentation and further reading

OpenAI Retrieval guide — semantic search, vector stores and chunking
<link href='https://developers.openai.com/api/docs/guides/retrieval' color='blue'>https://developers.openai.com/api/docs/guides/retrieval</link>

OpenAI embeddings guide — vector representations and similarity
<link href='https://developers.openai.com/api/docs/guides/embeddings' color='blue'>https://developers.openai.com/api/docs/guides/embeddings</link>

OpenAI File Search guide — hosted file search as an alternative to implementing retrieval yourself
<link href='https://developers.openai.com/api/docs/guides/tools-file-search' color='blue'>https://developers.openai.com/api/docs/guides/tools-file-search</link>

Hugging Face LLM Course — model and transformer concepts
<link href='https://huggingface.co/learn' color='blue'>https://huggingface.co/learn</link>